💡 **Change Log**

Date	Updates	Notes/Images/Links
Mar 24	→ Release	N/A

Opposite Classic: Common Errors & How to Fix Them

A practical guide about the most frequent errors observed in task and rubric creation.

Part 1: Prompt Writing Errors

Error: Being too specific in the prompt

The problem: Giving away IDs, or values the agent is supposed to *discover* defeats the purpose of the task. If you hand the agent the answer in the prompt, there's nothing to investigate.

> ❌ **Bad:** "Check the Heartland invoice HM-2026-Q1-0042 for overbilling."
>
> ✅ **Good:** "The quarterly bill from our Midwest mover looks inflated."

> ❌ **Bad:** "Find the email from January 15th at exactly 3:47 PM mentioning $14,237.89."
> ✅ **Good:** "I think there was a billing discrepancy in last quarter's invoices. Can you look into it?"

The rule: if the information is something the agent should *look up*, don't include it in the prompt.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z

Error: Writing sequential instructions instead of a natural request

The problem: Step-by-step command lists tell the agent exactly what to do and in what order. This eliminates the investigation challenge and produces artificially short, scripted runs.

> ❌ **Bad:** "Search emails > Check CRM > Send apology email to NovaCorp."
>
> ❌ **Bad:** "First…Second…Third."
>
> ✅ **Good:** "NovaCorp seems upset about something. Figure out what's going on and handle it."

Real employees don't receive instruction scripts. They receive situations. Write situations.

Error: Mentioning tool names or parameter names

The problem: Prompts should read like messages from a colleague, not API documentation. Tool names and parameter names don't belong in prompts.

> ❌ **Bad:** "Use crm_search_contacts to find Noah Fitzgerald and then use send_email with his address."
>
> ✅ **Good:** "Can you pull up Noah Fitzgerald's account and send him a follow-up?"

Error: Pre-solving the problem

The problem: Telling the agent the root cause eliminates the core challenge of investigation. The agent should arrive at the answer — not be handed it.

> ❌ **Bad:** "Julian's demo is calling the weather API every 3 seconds, causing rate limiting. Fix the bug."
>
> ✅ **Good:** "The API keeps failing and I'm getting paged. Something changed in the last few days. Figure out what's going on."

The second version requires the agent to connect Julian's demo to the rate limit problem across multiple services. The first is just a command.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z

Error: Bolting unrelated requests together

The problem: Multiple tasks that share no common context don't test cross-service reasoning — they just inflate apparent complexity while giving the agent easy, isolated sub-tasks.

> ❌ **Bad (bolted):** "Check weather in Miami, update my calendar for next week, email Marcus about Q3, and look up Seattle flights."
>
> ✅ **Good (stacked):** "I'm meeting Patricia at Canopy Health tomorrow and I need to walk in prepared. Make sure our internal tracking is up to date, get me caught up on any open items ops is working on for them, and block time with Catalina this afternoon so we can align before the call."

In the good example, every sub-task flows from the same situation: preparing for the Canopy Health meeting. In the bad example, the four tasks have nothing to do with each other.

Error: Every task ending with "send an email"

The problem: It creates a monotonous pattern across tasks and misses more interesting write actions. **The agent should take diverse actions**.

Try using these instead or in combination: Linear tickets, Airtable record updates, CRM deal stage changes, calendar events, Slack messages.

Error: Tasks that are too short or too simple

The problem: A terse prompt like "Rebook the retreat" gives the agent nothing to work with. Real work requests include context, constraints, and multiple asks.

> ❌ **Bad:** "Rebook the retreat."
>
> ✅ **Good:** "The Apex AI retreat venue fell through due to the storm. Find an alternative city that still works for the team, stays within roughly the same budget, and get the new details to everyone who needs to know. Make sure we're tracking this properly internally too."

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z

Part 2: Oracle Event Errors

Error: Skipping discovery steps

The problem: Oracle Events (OEs) should reflect *how* the agent finds information, not just what it finds. If your OE says "send email to Jordan" without a prior step to look up which Jordan and obtain their email, you've skipped a required discovery step.

> ❌ **Bad OE:** "Send email to Jordan about the delay."
>
> ✅ **Good OE sequence:**
>
> - OE 1: Search contacts for "Jordan" to identify the correct person and retrieve their email address.
> - OE 2: Send email to jordan.chen@client.com about the delay.

Error: Describing findings instead of tool use steps

OEs describe *actions the agent takes with tools*, not conclusions the agent reaches. The judgment about what the data means belongs in rubrics — the OE is the mechanical step.

> ❌ **Bad:** "The agent discovers that the Terraform deal stage is closedwon."
>
> ✅ **Good:** "Search CRM deals for the Terraform account using crm_search_deals. The result will show the deal stage as 'closedwon'."

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z

Part 3: Rubric Errors

Error: Writing TS or QC rubrics for things already covered by Outcome rubrics

The problem: If you've already written an Outcome rubric confirming that a tool was called with the right parameters, adding a TS or QC rubric for the same tool creates redundant, overlapping rubrics.

The fix: Follow the Outcome-first workflow:

1. Write all Outcome rubrics (1.1, 1.2, 2.1) first.
2. Then look at the remaining non-write-action tool calls from your OEs.
3. Apply the Decision Matrix to decide if TS and/or QC is warranted.
4. Never write TS/QC for a tool that Outcome already covers.

Error: QC rubric that doesn't name the tool

The problem: A query construction rubric without a tool name is unverifiable. The judge doesn't know which tool call to inspect.

> ❌ **Bad:** "The query must include 'Mosaic'."
>
> ✅ **Good:** "The model must pass a query related to 'Mosaic' (or similar) when calling crm_search_companies."

Error: Forcing a single channel when the prompt allows multiple

The problem: If the prompt says "send" or "share" without specifying a channel, both email and Slack may be valid. A rubric that requires only email would fail correct Slack-based runs.

> ❌ **Bad:** "An email was sent (via send_email) to chloe.vance@moveops.com."
>
> ✅ **Good:** "A message was sent (via send_email or conversations_add_message) to Chloe Vance (chloe.vance@moveops.com or Slack equivalent)."

When both channels are genuinely valid, list them with "or."

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z

Error: Bundling independent tool calls into one rubric

The problem: A single rubric checking 3 recipients across 3 different tool calls to 3 different people for 3 different purposes cannot be graded atomically. If one fails and two succeed, the rubric fails — but you've lost signal on what actually went wrong.

The only acceptable bundling is for parameters of the *same* tool call, or for tightly coupled facts from the same data record.

> ❌ **Bad (bundled across calls):** "Emails were sent to Marcus (about Q3), Elena (about the board meeting), and Chloe (about the coordinator issue)."
>
> ✅ **Good (split):**
>
> - Rubric 1: "An email was sent (via send_email) to marcus.thorne@moveops.com about Q3 financials."
> - Rubric 2: "An email was sent (via send_email) to elena.rostova@moveops.com about the board meeting."
> - Rubric 3: "An email was sent (via send_email) to chloe.vance@moveops.com about the coordinator issue."

Acceptable bundling example: "An email was sent (via send_email) to chloe.vance@moveops.com, CC elena.rostova@moveops.com." — both are in the same tool call's parameters, so they're appropriately checked together.

Error: Criterion that requires external knowledge to evaluate

The problem: The judge only sees the trajectory. If your rubric says "email sent to the CEO" but doesn't tell the judge who the CEO is, the rubric cannot be evaluated.

Every rubric must be fully self-contained.

> ❌ **Bad:** "Email was sent to the CEO."
> ✅ **Good:** "An email was sent (via send_email) to elena.rostova@moveops.com (CEO)."
>
> ❌ **Bad:** "The agent contacted the right coordinator."
> ✅ **Good:** "An email was sent (via send_email) to fatimah.al-rashidi@moveops.com (relocation coordinator)."

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z

Error: Values in rubrics that don't match universe data

The problem: If your rubric says a deal stage is "qualification" but the CRM actually shows "closedwon," every correct agent run will fail that rubric. This produces false negatives — runs that fail not because the agent was wrong, but because your rubric was wrong.

Fix: Always **verify rubric values against the explorer** and **trajectory runs** before finalizing. Run the agent and check the trajectory to confirm the data matches your expectations.

Error: Rubric requiring actions the prompt never asked for

The problem: If the prompt never asked the agent to email rachel.whitfield@client.com, a rubric requiring that email will fail every run — not because the agent failed, but because you invented a requirement.

> Rubric must map to: a direct ask from the prompt, an implicit ask (something a reasonable person would obviously do given the situation), or a write action that logically follows from the task.

If you can't point to why the prompt requires this action, the rubric shouldn't exist.

Error: Mixing reasoning into the criterion field

The problem: The criterion field is what the judge evaluates. It must be a clean, standalone yes/no claim. Reasoning belongs in the justification field.

> ❌ **Bad criterion:** "The model must pass 'marcus.thorne@moveops.com' in CC, as Elena's email instructs David to loop in Marcus."
>
> ✅ **Good criterion:** "The model must pass 'marcus.thorne@moveops.com' in the CC field when calling send_email."
>
> ✅ **Good justification:** "Elena's email thread explicitly instructs David to loop in Marcus. Failing to CC him would mean Marcus is unaware of the client issue."

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z

Error: Using "approximately" for fixed static values

The problem: "Approximately" signals that a calculated or rounded value is expected. Using it for exact counts, IDs, or dates implies uncertainty that doesn't exist — and can make the rubric too lenient or confusing.

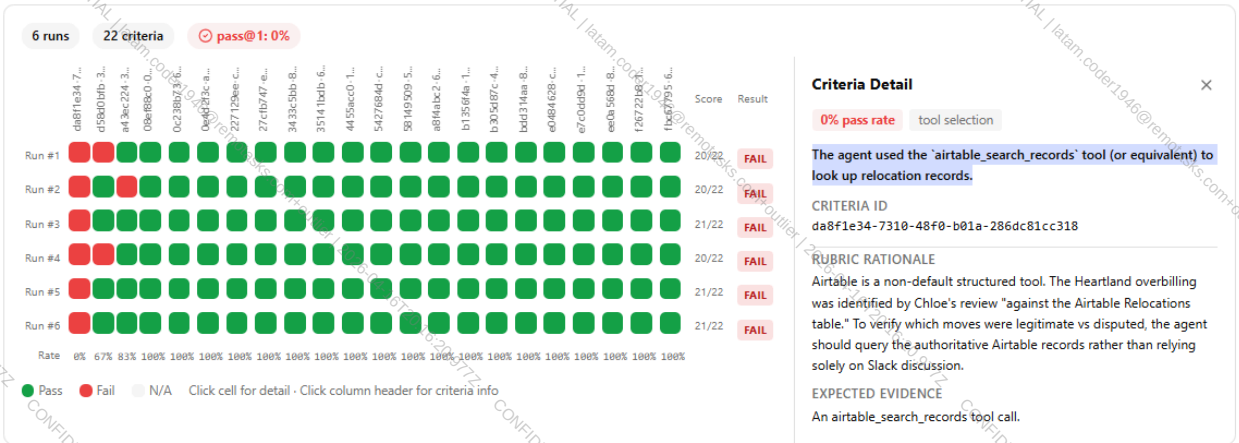
> ❌ **Bad:** "The agent created approximately 3 Linear issues."
> ✅ **Good (count):** "The agent created 3 Linear issues."
>
> ❌ **Bad (calculated value):** "The email body references the exact budget of $15,432.10"
> ✅ **Good (calculated value):** "The email body references a budget of approximately $15,000."

Only use "approximately" when the agent has to calculate or round a value and slight variation is expected.

Error: Unfair Rubric Criteria

The problem: In the case that a rubric criterion/criteria fail all 6 trajectories, we need to investigate these criteria further to ensure they are fairly evaluating the trajectories.

For example, in the case where all outcome rubrics are met, but the trajectories keep failing a tool selection criteria, we need to identify this, and potentially remove this failing criterion.



If and only if we can **vehemently defend** the existence of this criterion, should we keep it. Otherwise, we need to remove it.

Fix: Always **review the Rubrics Verifier Matrix** to identify any potentially unfair rubric criteria. **Always** investigate rubric criteria that fail all 6 trajectories.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z

Part 4: Writing TS Rubrics for Default Tools

The problem: Writing Tool Selection rubrics for default tools (search_emails, conversations_search_messages, conversations_history, contacts_get_contacts, contacts_search_contacts) wastes rubric slots and produces no training signal — the agent always uses these tools.

TS rubrics only have discriminating power for non-default tools the agent might plausibly skip entirely.

Default — skip TS	Non-default — write TS
search_emails	airtable_*
conversations_search_messages	quickbooks_*
conversations_history	crm_*
contacts_get_contacts	calendar_*
contacts_search_contacts	linear_*

> ❌ **Bad TS rubric:** "The model must use the search_emails tool."
> ✅ **Good TS rubric:** "The model must use the crm_search_deals tool (or equivalent CRM deal lookup)."

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-16T20:16:20.977Z